Photometric Stereo Web Viewer — with Python

Turn the outputs of a photometric stereo recording into full-resolution JPEGs and a deep-zoom web viewer with two Python scripts. Everything runs from the command line, so it is easy to repeat on many pieces.

The viewer shows three perfectly aligned layers of the same piece: the **albedo** (surface colour without shadows), the **relief** (shaded surface) and the **overlay** of both. It is built with OpenSeadragon and opens with a double-click.

What you need

- Python 3.9 or newer (tested with 3.13) on Windows, macOS or Linux.
- The colour albedo, usually a 16-bit TIFF.
- The relief, in either form:
  - the **shaded relief exported by Selene** (PNG or TIFF), which is used as it is;
  - or a **32-bit height map** (floating-point TIFF), which the script shades itself.
- Albedo and relief must have exactly the same size in pixels.
- An internet connection when opening the viewer, because it loads OpenSeadragon from a CDN.

What is in this folder

```
with_python/
  data/                 put your albedo and relief here
  images/               albedo.jpg, relief.jpg, overlay.jpg (created by process.py)
  viewer/
    viewer.html         the full viewer
    viewer_minimal.html the shortest possible viewer, for teaching
    tiles/              created by make_tiles.py
  process.py            makes the three JPEGs
  render_relief.py      example: renders a shaded relief from a depth map
  make_tiles.py         cuts them into tiles for the viewer
  requirements.txt      Python packages needed
```

Step 1. Install the dependencies (only once)

1. Install Python from python.org. On Windows, tick **Add Python to PATH** in the installer.
2. Open a terminal in this folder and run:

```
pip install -r requirements.txt
```

This installs numpy, tifffile, Pillow and pyvips. The `pyvips[binary]` package already includes vips, so nothing else is needed on Windows, macOS or Linux.

Step 2. Make the three images

1. Copy the albedo and the relief into `data`.
2. Run `process.py` with both files:

```
python process.py data\albedo_m1.tif data\depthmap_m1_shaded.png
```

3. Check the result in `images`: `albedo.jpg`, `relief.jpg` and `overlay.jpg`, at full resolution.

What it does:

- **Albedo:** converts the 16-bit image to 8 bits.
- **Relief:** if it is Selene's shaded relief, it is saved as it is; if it is a 32-bit height map, the script computes a hillshade with the same formula as GIS tools such as gdaldem.
- **Overlay:** multiplies the albedo by the relief, with the same result as Multiply at 60% in Photoshop.

A 75-megapixel piece takes about 20 seconds.

Options you can add:

Option	Default	What it does
`--strength`	0.6	How much the relief darkens the overlay, from 0 to 1
`--stretch`	off	Expands the albedo contrast between the 0.1 and 99.9 percentiles
`--gamma`	1.0	Use 2.2 if the albedo is in linear space
`--quality`	95	JPEG quality
`--azimuth`	45	Height maps only: light direction in degrees, clockwise from the top (45 = from the top right)
`--altitude`	45	Height maps only: light elevation; lower values give more raking light
`--exaggeration`	1.0	Height maps only: vertical exaggeration of the relief
`--pixel-size`	1.0	Height maps only: pixel size in the same unit as the heights, e.g. 0.05 for 0.05 mm

Example with a height map in millimetres and a pixel of 0.05 mm:

```
python process.py data\albedo.tif data\heights.tif --pixel-size 0.05 --exaggeration 3
```

If the computed relief looks flat, raise `--exaggeration` or check `--pixel-size`. JPEG cannot exceed 65,500 px on either side.

Example: render the relief from a depth map

Instead of Selene's shaded image, you can start from the depth map (the integrated surface heights) and render the shaded relief yourself. This lets you choose the light, the vertical exaggeration and the units, and compare several lights.

1. Copy the depth map into `data`. It can be a 32-bit floating-point TIFF, or a 16-bit TIFF or PNG.
2. Render it with the standard light, from azimuth 45° (top right) at 45° elevation:

```
python render_relief.py data\depthmap_m1.tif
```

3. Open the result, `data\depthmap_m1_render.png`. The script also prints the median slope of the surface; if the relief will look flat, it suggests an `--exaggeration` value to try:

```
python render_relief.py data\depthmap_m1.tif --pixel-size 0.05 --exaggeration 12
```

4. Use the render as the relief in Step 2:

```
python process.py data\albedo_m1.tif data\depthmap_m1_render.png
```

To compare raking light from several directions, give several azimuths. Each one is saved as a separate PNG ending in `_az45`, `_az135` and so on:

```
python render_relief.py data\depthmap_m1.tif --azimuth 45 135 225 315
```

Option	Default	What it does
`--azimuth`	45	Light direction in degrees, clockwise from the top of the image; several values make several renders
`--elevation`	45	Light elevation in degrees; lower values give more raking light
`--exaggeration`	1.0	Vertical exaggeration
`--pixel-size`	1.0	Pixel size in the same unit as the depths, e.g. 0.05 for 0.05 mm
`-o`	next to the input	Output PNG file

You can also give the depth map straight to `process.py`, which recognises 32-bit files and renders them with the same light. `render_relief.py` is useful to check and adjust the render before making the overlay.

Step 3. Make the tiles

```
python make_tiles.py
```

It cuts the three images into Deep Zoom tiles inside `viewer\tiles`, in a few seconds, and writes the image size into the viewer so there is nothing to edit by hand. Running it again rebuilds the tiles.

Step 4. Open the viewer

1. Double-click `viewer\viewer.html`. No server is needed.

2. To show the name of the piece, open `viewer.html` in a text editor and change `title` in the `CONFIG` block near the end.

`viewer_minimal.html` is the shortest possible OpenSeadragon viewer: a single line pointing to `tiles/albedo.dzi`. Browsers do not let a page read `.dzi` files from the local disk, so open it through a local server, which is also a good moment to explain how web servers work:

```
cd viewer
python -m http.server 8000
```

Then go to `http://localhost:8000/viewer_minimal.html` in the browser.

Using the viewer

The viewer has four modes, chosen with the buttons at the bottom of the screen or with the number keys 1 to 4.

Mode	Key	What you see	Use it to
Albedo	1	The surface colour, without shadows	Study pigments, inks, inscriptions and colour variations
Relief	2	The shaded surface, in grey	Read tool marks, brushstrokes, plate marks, cracks and texture
Overlay	3	Colour and relief combined	Get a natural, readable image that shows both at once
Compare	4	Albedo with the relief faded on top	Relate a colour feature to the surface shape beneath it

1. Start in **Albedo**. Zoom with the mouse wheel or a pinch, and drag to move around.
2. Stop on an area of interest and press **2** to switch to **Relief** without losing your position.
3. Press **3** to see the **Overlay**.
4. Press **4** for **Compare** and move the slider between Albedo and Relief to fade one into the other.
5. Use the minimap in the bottom-right corner to see where you are and jump to another area.
6. Use the buttons in the top-right corner to zoom in, zoom out, return to the full view or go full screen.

All layers are preloaded, so switching between modes is instant. On phones the minimap is hidden so it does not cover the controls.

Publishing the viewer

The `viewer` folder is a static website: upload it as it is and it works, with nothing to install on the server.

1. Check that the viewer works on your computer first.
2. Upload the whole `viewer` folder, including `tiles`, keeping the same structure.
3. Open the address of `viewer.html` on the server.

Good options are GitHub Pages (free, for public projects), any Apache or Nginx server, or your institution's web space. The tile folders contain thousands of small files, so an FTP client is faster than uploading through a web form.

To embed the viewer in an existing web page, use an `iframe`:

```
<iframe src="viewer/viewer.html" width="100%" height="600" allowfullscreen></iframe>
```

Troubleshooting

Problem	Likely cause	Fix
`ModuleNotFoundError: No module named 'pyvips'` (or numpy, tifffile, PIL)	The dependencies are not installed	Run `pip install -r requirements.txt`
`python` is not recognised	Python is not in the PATH	Reinstall Python ticking **Add Python to PATH**, or use `py` instead of `python`
The viewer shows "The tiles could not be loaded"	`make_tiles.py` has not been run, or `images` is missing a file	Run `process.py` and `make_tiles.py` again
`viewer_minimal.html` stays grey after a double-click	The browser blocks `.dzi` files from the local disk	Open it through `python -m http.server`, as in Step 4
`Size mismatch`	Albedo and relief have different sizes	Export both at the same resolution
The albedo looks too dark and flat	It is stored in linear space	Add `--gamma 2.2`, or `--stretch`
The computed relief looks flat	Height units and pixel size do not match	Set `--pixel-size` and raise `--exaggeration`
`MemoryError`	Very large piece for the available RAM	Close other programs; roughly 4 GB of RAM per 100 megapixels is needed
The viewer buttons have no icons, or nothing appears	No internet connection	OpenSeadragon loads from the internet; connect, or download it and point the `script` tag to the local copy

Credits

Developed by Factum Foundation for the AOA-2026 workshop. The viewer uses OpenSeadragon and the tiles are made with libvips.